유창헌

FRONTEND DEVELOPER · 4년차

불편을 발견하면,
직접 해결합니다.

실시간 통신과 개발자 도구에 관심이 많은 프론트엔드 개발자입니다. React, JavaScript, TypeScript를 주로 다루지만, 해결이 필요한 곳이라면 기술 스택에 국한되지 않고 해결합니다. 최근 AI를 적극적으로 사용하여 익숙한 스택의 경계에 덜 갇히고, 필요하면 새로 배워가며 만듭니다. 아래 프로젝트는 제가 직접 겪은 불편을 도구로 만든 것들입니다.

3

직접 배포·운영 중인 제품
웹스토어 · npm · vsix

367회

시세 위젯 누적 다운로드
사용자 53명 · 평점 5.0

---

github.com/dbckdgjs369 dbckdgjs1130@naver.com 010-2809-6995

	개인 프로젝트 3종	전부 배포·운영 중
P1	`Stock Price Widget`	스토어 평점 5.0 · 운영 중
P2	`react-code-locator`	npm v1.0.2 · 번들러 6종 실측
P3	`Custom Tunnel`	빌드·배포 120초 → 0 · 실무 적용
P4	`CC Talk`	매일 쓰는 도구 · TUI 스크래핑 → 헤드리스 stream-json

P1 · Chrome Extension + Node.js 중계 서버 · 2026.03 — 진행중 · v4.1.1

Stock Price Widget

어떤 웹페이지 위에서든 주식·코인 시세를 실시간으로 띄우는 위젯 오버레이입니다. 위젯 아래 페이지는 그대로 클릭되고, 우클릭 한 번으로 직접 만든 캔들 차트가 열립니다.

스토어 평점 5.0 | 운영 중 | 자체 캔들 차트 엔진 | 캐시·프리페치로 전환 지연 0 | 국내·해외주식·코인

Chrome 웹스토어 — https://chromewebstore.google.com/detail/stock-price-widget/niofpnhhdeaneajeemfpnclammllkhfg

---

왜 만들었나

주식·코인 시세가 수시로 궁금한데, 확인하려면 그때마다 보던 화면을 벗어나 증권사 앱이나 거래소 탭을 따로 열어야 했습니다. 돌아오면 하던 일의 맥락이 끊겨 있었습니다.

"지금 보고 있는 이 페이지 위에 시세가 그냥 떠 있으면 좋겠다"는 생각에서 시작했습니다. 조건은 두 가지였습니다 — 위젯이 일하는 걸 방해하지 않을 것(아래 페이지가 그대로 클릭될 것), 그리고 실제로 실시간일 것(새로고침해야 갱신되는 건 의미가 없음).



확장 팝업 (한국어) 생성을 누르면 보고 있던 페이지에 위젯이 바로 붙습니다. 코인은 거래소·현물/선물을 좁혀 찾을 수 있습니다.

어떻게 만들었나

1. 설치 직후, 안내를 띄울 화면 자체가 없었다

문제 이 확장은 보고 있는 페이지 위에 위젯을 얹는 게 전부입니다. 그런데 설치를 마친 직후 사용자가 보고 있는 탭은 대개 크롬 웹스토어이고, 크롬 정책상 스토어와 chrome:// 페이지에는 확장 스크립트가 주입되지 않습니다. 그 화면에서는 위젯도, 안내도 아무것도 뜰 수 없습니다. 설치 → 아무 일도 안 일어남 → 이탈, 이게 획득 단계에서 가장 크게 새는 구멍이었습니다.

선택 설치 시점에 확장이 소유한 페이지(welcome.html)를 직접 열어 착륙 지점을 만들었습니다. 확장 자기 페이지는 주입이 아니라 일반 로드라 정책 제약을 받지 않아서, 여기서 안내를 시작할 수 있습니다. 이어서 팝업 → 실제 웹페이지로 3막에 걸쳐 넘깁니다 — 각 막은 스크립트가 확실히 살아 있는 화면에서만 돕니다. 제약을 숨기지 않았습니다 — 이 페이지에 「여기서는 위젯이 동작하지 않습니다」를 그대로 띄우고, 바로 열 수 있는 일반 페이지 두 개를 버튼으로 줬습니다. 「왜 안 되지」로 이탈하는 것보다 「저기로 가면 된다」가 낫습니다 임의 URL 에 안내를 주입하는 길은 버렸습니다 — 그러려면 <all_urls> 권한을 요청해야 하고, 그러면 설치 화면에 "모든 웹사이트의 데이터를 읽고 변경" 경고가 붙습니다. 획득이 병목인 확장에서 설치 문턱을 높이는 건 남는 장사가 아닙니다 툴바 고정 안내에는 화살표를 쓰지 않았습니다 — 퍼즐 아이콘은 브라우저 UI라 어떤 문서의 DOM 에도 없고, 좌표가 고정 확장 개수·프로필 아바타·창 컨트롤·확대율에 따라 다 움직입니다. 예전엔 innerWidth - 240 을 가리키는 화살표를 뒀는데 대개 프로필 쪽을 찍었습니다. 틀린 화살표는 없는 화살표보다 나쁩니다. 카드를 우상단에 붙여 방향만 전합니다

결과 설치 직후 빈 화면을 보는 경로가 없어졌습니다. 안내는 스크립트가 반드시 동작하는 화면에서만 시작하고, 최초 설치에만 한 번 뜹니다. 이 페이지의 문구도 한국어·영어 모두 _locales에서 옵니다 — 언어에 따라 위키백과 링크까지 각 언어판으로 바뀝니다.

2. 튜토리얼이 사용자 손이 움직이는 중에 넘어갔다

문제 설치 직후 실제 UI 위에 스포트라이트를 띄우는 온보딩 투어를 만들었습니다. "위젯을 끌어서 옮겨 보세요" 처럼 사용자 동작을 기다리는 단계가 있는데, 이걸 pointerdown 이나 폴링으로 판정하면 아직 끌고 있는 중에 다음 단계 문구로 바뀌어 화면이 흔들립니다. 반대로 느슨하게 두면 제자리에서 한 번 클릭만 해도 단계가 넘어가, 배우지 않고 통과합니다.

선택 판정을 손을 뗀 시점(pointerup)으로 옮기고, "실제로 했다"의 기준을 이동량으로 정의했습니다. 이동 6px(맨해튼 거리) 미만은 제자리 클릭으로 보고 넘기지 않음 너비 변화 12px 미만은 크기 조절로 인정하지 않음 — 시세 갱신 리렌더로 생기는 1~2px 흔들림을 조작으로 오인하면 안 됩니다 12초 교착 타이머 — 위젯 상한에 걸려 버튼이 disabled 인 것처럼 조작 자체가 불가능한 상황에서는 아무리 기다려도 조건이 성립하지 않습니다. 일정 시간이 지나면 수동 "다음" 버튼을 열어 갇히지 않게 했습니다

검증 확장을 실제 Chromium 에 로드해 Shift+드래그·마퀴 선택·우클릭까지 재현하는 E2E 를 짰습니다. 중요한 건 음성 검증입니다 — 제자리 클릭이나 4px 리사이즈로는 단계가 넘어가지 않아야 합니다. 성공 경로만 보면 판정이 느슨해지는 회귀를 못 잡습니다.

결과 끌고 있는 동안 화면이 흔들리지 않고, 클릭만으로 단계를 건너뛰는 것도 막혔습니다. 최초 설치에만 자동 실행되고, 다시 보고 싶으면 팝업에서 재생할 수 있습니다.

3. API 키를 확장에 넣을 수는 없다

문제 크롬 확장은 사용자 컴퓨터에 소스가 그대로 설치됩니다. 증권사 API 키를 확장에 넣으면 누구나 꺼내 볼 수 있습니다.

선택 시세 공급자 연결을 통째로 Node.js 중계 서버로 옮겼습니다. 확장은 서버의 API와 실시간 채널만 사용하고, 키는 서버 환경변수에만 존재합니다. 덤으로 같은 종목을 여러 사용자가 봐도 상류 연결은 하나로 두고 fan-out 하게 되어, 증권사 API 호출 수도 줄었습니다.

결과 키가 노출되지 않고, 공급자를 바꿔도 확장은 손대지 않습니다. 실제로 시세 소스 구조를 서버로 옮기는 동안 확장 코드는 그대로였습니다.

4. 로컬에선 압축이 되는데 프로덕션에서만 안 됐다

문제 시세 push 는 clientId(UUID 36B)·종목명·통화·거래소명이 매 틱 반복되는 고중복 JSON입니다. 압축 없이는 월 전송량이 27.8GB까지 올라가는 구조였고, 아래에서 보듯 표준 압축을 넣었는데도 프로덕션에서만 대역폭이 줄지 않았습니다.

원인 서버(Render) 앞단의 Cloudflare 가 압축 확장 헤더를 제거하고 있었습니다. 헤더가 사라지면 협상이 성립하지 않고, 구현상 조용히 무압축으로 떨어지도록 되어 있어 에러도 로그도 남지 않았습니다. 제 코드가 아니라 제어할 수 없는 중간 계층이 원인이었습니다.

실패 먼저 만든 건 표준 압축(permessage-deflate)이었습니다. 협상·프레이밍·zlib 문맥 관리까지 직접 구현했고, 통합 테스트도 통과했습니다. 그런데 프로덕션에서만 압축이 걸리지 않았습니다. 핸드셰이크를 찍어보니 이렇게 갈렸습니다. // 로컬·직결 — 브라우저 제안이 그대로 도착 → Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits ← Sec-WebSocket-Extensions: permessage-deflate; client_no_context_takeover negotiate(offer) → { windowBits: 12, … } 프레임 RSV1 = 1 → 압축됨 // Cloudflare 뒤 — 제안 헤더 자체가 사라진 채 도착 → (Sec-WebSocket-Extensions 없음) ← (응답에도 없음) negotiate(undefined) → null 프레임 RSV1 = 0 → 무압축 협상이 성립하지 않으면 무압축으로 조용히 되돌아가도록 설계해 둔 게 여기서 발목을 잡았습니다. 규약상 맞는 처리지만, 그래서 에러도 로그도 남지 않았습니다. 게다가 테스트는 서버에 직접 붙으니 프록시를 통과하는 경로만 검증되지 않았습니다. 그다음으로 "문맥 없이 메시지별로 압축하면?"도 재봤지만, 페이로드가 300B 미만이라 -24.9%에 그쳐 포기했습니다.

선택 프록시가 손댈 수 있는 층(압축 확장)에 의존하지 않고, 건드릴 수 없는 층 — JSON 자체를 줄이기로 했습니다. 구현 전에 scripts/measure-ws-compression.mjs 로 4개 안을 실측해 비교했습니다.

측정 메시지 20,000개 기준 평균 크기 절감 무압축(당시) 274.8B 기준 독립 압축(문맥 없음) 206.2B -24.9% 페이로드 다이어트 64.8B -76.4% 다이어트 + 문맥 유지 압축 20.5B -92.5% 페이로드가 300B 미만이라 메시지별 독립 압축은 25% 밖에 못 줄입니다. 연결 단위로 압축 문맥을 유지해야(context takeover) 90%대가 나오는데, 그게 바로 프록시에 막힌 방식이었습니다.

구현 프로토콜 v2 — 한 번 받으면 클라이언트가 이미 아는 clientId 를 빼고, 아무도 쓰지 않는 필드를 걷어내고, 종목별로 고정된 통화·거래소명은 바뀔 때만 보냅니다. 시세는 객체 대신 배열 튜플로 보냅니다. 고정값을 생략하니 인코더가 연결별 캐시를 갖게 되어, 클라이언트도 같은 캐시로 빠진 필드를 복원합니다. 이미 배포된 구버전 확장이 깨지지 않도록 ?pv= 버전 협상을 두고, 요청이 없으면 v1 그대로 응답합니다.

예시 삼성전자 시세 한 틱이 실제로 이렇게 바뀝니다. // v1 — 274B {"event":"quote","payload":{ // UUID 36B, 매 틱 반복 — 클라가 이미 안다 "clientId":"6f1d2c4a-9b3e-4f70-8a21-5c7e93d0b8f1", "quotes":[{ "symbol":"005930", "shortName":"삼성전자", // 안 쓰는 필드 "price":71200, "change":1200, "changePercent":1.71, // 심볼당 고정값 — 바뀔 때만 보내면 된다 "currency":"KRW", "exchangeName":"KRX", "marketState":"TRADING", // 안 쓰는 필드 "updatedAt":1742188800000 }] }} // v2 — 56B {"e":"q","p":[["005930",71200,1200,1.71,1742188800000]]} // e = 이벤트("q" = quote) // p = [[심볼, 현재가, 등락, 등락률, 시각]] 274B → 56B (-79.6%) — 같은 정보를 그대로 복원할 수 있습니다. clientId 는 연결 시 ready 로 한 번 받아 클라이언트가 이미 알고, 통화·거래소명은 바뀔 때만 튜플 끝에 붙습니다. 첫 전송과 재연결 직후에는 ["KRW","KRX"] 가 실려 70B가 되고, 그걸로 클라이언트 캐시를 세웁니다.

결과 평균 274.8B → 64.8B (-76.4%), 월 전송량 27.8GB → 6.5GB. 남은 과금분은 이후 서버를 무료 티어 인프라로 옮겨 0으로 만들었습니다. 압축 구현은 그대로 남겨 뒀습니다 — 프록시를 거치지 않는 경로에서는 두 절감이 함께 걸려 -92.5%가 됩니다. 인프라가 협조하지 않아도 확보되는 하한선을 갖게 된 셈입니다.

5. 위젯이 페이지를 가로막지 않게

문제 페이지 위에 요소를 띄우면 그 아래 링크·버튼이 클릭되지 않습니다. 시세를 보려고 일을 못 하게 되면 안 됩니다. 게다가 남의 사이트 CSS가 위젯 스타일을 덮어쓰는 문제도 있었습니다.

선택 Shadow DOM으로 스타일을 격리해 어떤 사이트에서도 위젯이 동일하게 보이게 하고, pointer-events를 조정해 위젯 영역의 클릭이 아래 페이지로 통과하도록 했습니다. 배경색 대비도 자동 보정해 밝은 사이트든 어두운 사이트든 읽히게 했습니다.

결과 시세를 띄워둔 채로 원래 하던 일을 그대로 할 수 있습니다.

6. 단축키가 브라우저 기본 동작과 충돌했다

문제 초기 설계에서 Shift + 드래그를 사각형 범위 선택에 썼는데, 이게 브라우저 기본 동작인 텍스트 범위 선택과 충돌했습니다. 대안으로 넣은 Cmd + 클릭 개별 선택은 클릭 통과를 위해 걸어둔 pointer-events: none 때문에 아예 동작하지 않았습니다. 5번에서 얻은 장점이 그대로 부작용이 된 셈입니다.

선택 단축키 체계를 재설계했습니다. 범위 선택을 Cmd·Ctrl + 드래그로 옮겨 텍스트 선택과의 충돌을 없애고, Shift + 드래그는 이동·크기 조절 전용으로 정리했습니다. pointer-events 처리도 고쳐 개별 선택이 정상 동작하게 했습니다.

결과 선택·이동·그룹 이동이 브라우저 기본 동작과 부딪히지 않고 자연스럽게 갈라집니다. (v3.4.2)

7. 시세가 옛날 값에 멈춰 있었다

문제 노트북을 덮었다 열거나, 와이파이가 바뀌거나, 서버가 재시작되면 시세가 갱신을 멈추고 옛 값에 그대로 머물러 있었습니다. WebSocket이 끊겼는데 양쪽 다 끊긴 줄 모르는 half-open 상태여서, 오류도 안 나고 재연결도 트리거되지 않았습니다. 가장 위험한 종류의 버그였습니다 — 화면에 숫자가 멀쩡히 떠 있어서 사용자가 틀린 줄 모릅니다.

선택 연결이 살아 있는지를 주기적으로 확인해 half-open을 감지하고, 끊긴 것으로 판단되면 자동 재연결하도록 했습니다.

결과 절전·네트워크 전환·서버 재시작 후에도 스스로 복구합니다. (v3.6.0)

8. 차트를 직접 만든 이유

문제 차트 라이브러리를 넣으면 번들이 커지고, 남의 페이지 위에 얹히는 오버레이 안에서 스타일·이벤트가 충돌하기 쉽습니다. 실시간 시세를 캔들에 밀어 넣는 방식도 라이브러리마다 제약이 달랐습니다.

선택 캔들 차트 엔진을 직접 구현했습니다. 캔들·거래량 렌더링, 주기 전환, 지표(MA·EMA·볼린저밴드), 드로잉 도구(추세선·레이·수평선·피보나치 되돌림)와 자석 스냅까지 만들었습니다. 캔들 몸통·심지·거래량 막대는 픽셀 정렬을 직접 잡아 안티앨리어싱 번짐과 좌우 비대칭을 없앴습니다.

결과 v4.0.2에서 차트를 전체 무료 공개했습니다. 성능·스타일을 오버레이 환경에 맞게 통제할 수 있게 됐습니다.

9. 체감 속도를 만든 3가지

문제 1분봉 → 일봉처럼 주기를 바꿀 때마다 매번 API를 다시 호출하니 로딩 스피너가 뜨고 멈칫했습니다. 차트를 보는 동작은 원래 주기를 여러 번 왕복하는 일인데, 그때마다 기다리게 되니 흐름이 끊겼습니다. 반대로 빠르게 연달아 누르면 먼저 보낸 요청이 나중에 도착해 이미 바뀐 화면을 덮어썼습니다 — 일봉을 보고 있는데 갑자기 1분봉이 그려지는 식입니다.

선택 전부 "빠르게 느껴지는가"라는 한 문제인데 층이 달라서, 세 군데에서 나눠 풀었습니다. 이미 본 주기 — 2단 캐시 + stale-while-revalidate. 메모리(Map, LRU 40개) → chrome.storage.local(40키) 순으로 찾고, 전환 시엔 신선도를 따지지 않고 캐시를 먼저 그린 뒤 백그라운드로 재검증합니다. TTL은 주기마다 다르게 뒀습니다 — 1분봉 30초, 15분봉 5분, 4시간봉 30분, 일봉 10분, 주·월봉 1시간. 갱신 주기가 다른 데이터에 같은 TTL을 주면 한쪽은 낭비, 한쪽은 낡기 때문입니다 아직 안 본 주기 — 순차 프리페치. 캐시는 한 번 본 주기만 빨라지니, 처음 열고 다른 주기를 누르면 여전히 기다려야 했습니다. 차트를 연 뒤 1.2초 후부터 나머지 주기를 300ms 간격으로 미리 채웁니다. 지연은 첫 화면 렌더와 요청이 겹치지 않게, 순차는 6개를 동시에 던져 API를 때리지 않게 하려는 것입니다. 실패는 무시하고, 차트가 닫히면 즉시 중단합니다 늦게 도착한 응답 — 그냥 버린다. 앞의 둘이 "빠르게 만드는" 쪽이라면 이건 빠른 조작을 견디게 하는 쪽입니다. 그리기 직전에 "지금도 이게 맞나"를 확인하는 가드(stillCurrent())를 둬, 요청 시점의 주기·종목·차트 존재 여부가 하나라도 달라졌으면 렌더를 버립니다. 받아온 데이터는 캐시에 그대로 저장하니 버린 요청도 낭비가 아니고, 차트를 닫은 뒤 도착한 응답이 없는 요소를 건드리던 문제도 같이 사라졌습니다

결과 주기 전환에서 체감 지연이 사라졌고, 캐시가 빈 첫 전환부터 즉시 그려집니다. 아무리 빠르게 눌러도 마지막으로 고른 주기만 화면에 남습니다. 탭을 오래 켜둬도 LRU 상한으로 메모리가 무한히 늘지 않습니다.

10. 과거를 보려면 드래그하다 끝에서 막혔다

문제 차트를 왼쪽으로 끌어 과거를 보다 보면 처음 받아온 구간 끝에서 더 이상 안 갔습니다. 과거를 더 보려면 별도 조작이 필요했는데, 사용자는 그냥 계속 끌고 싶어 합니다.

선택 뷰포트 왼쪽 끝에서 10개 캔들 이내로 접근하면 과거 데이터를 자동 조회해 앞에 이어붙입니다. 여기서 챙긴 것들: 중복 요청 차단 — 엔진·호출부 양쪽에 로딩 플래그를 둬 드래그 중 같은 구간을 여러 번 받지 않게 함 중복 캔들 제거 — 기존 시각을 Set으로 모아 겹치는 캔들을 걸러냄. 경계에서 캔들이 겹쳐 들어오면 그래프가 틀어짐 뷰포트 보정 — 앞에 n개를 붙이면 인덱스가 밀리므로 보고 있던 구간을 n만큼 이동시켜 화면이 튀지 않게 함 종료 처리 — 빈 응답이 오면 "더 없음"으로 표시해 이후 요청을 멈춤 타임존 보정 — 분봉은 서버가 UTC 기준이라 조회 기준 시각에 오프셋을 적용

결과 끊김 없이 계속 끌어서 과거를 볼 수 있습니다. 지표와 드로잉도 데이터가 늘어날 때 함께 다시 계산됩니다.

결과

- 기획 · 프론트 · 중계 서버 · 배포 · 운영까지 혼자 끌고 가 크롬 웹스토어에 출시.
- 누적 다운로드 367회 · 사용자 53명, 스토어 평점 5.0을 유지하며 운영 중.
- 출시 후에도 v4.1.1까지 이어서 개발 — 차트 무료 공개, 위젯 한도 확대, 연결 안정화, 단축키 재설계.
- 운영 비용까지 직접 관리했습니다 — 프록시가 압축을 무력화한 걸 찾아내 WS 프로토콜을 다시 설계, 월 전송량 27.8GB → 6.5GB(-76%)로 줄이고, 이후 무료 티어 인프라 이전까지 겹쳐 운영 비용을 0으로 만들었습니다.
- 실사용에서 나온 문제(끊긴 연결·단축키 충돌·해외 시세 파싱 오류)를 버전을 올려가며 해결.
- 이탈이 나는 지점을 제품으로 막았습니다 — 스토어 페이지에는 확장 스크립트가 주입되지 않아 설치 직후엔 위젯도 안내도 뜰 수 없었습니다. 확장 소유 페이지를 착륙 지점으로 만들고 3막 온보딩으로 실제 웹페이지까지 데려갑니다.
- 회귀를 잡는 테스트를 남겼습니다 — 확장을 실제 Chromium 에 로드해 단축키·드래그·우클릭을 재현하고, "넘어가지 않아야 한다"는 음성 검증까지 넣었습니다. 성공 경로만 보는 테스트는 판정이 느슨해지는 걸 놓칩니다.
- 차트는 기능을 붙인 뒤 체감 속도까지 손봤습니다 — 2단 캐시·프리페치로 주기 전환 지연을 없애고, 늦게 온 응답이 화면을 덮지 않게 가드를 두고, 드래그만으로 과거가 계속 이어지게 했습니다.

P2 · npm package · 빌드 플러그인 · 2026.03 — 진행중 · v1.0.2

react-code-locator

개발 중 React 화면에서 UI를 Shift를 누른 채 우클릭하면 그 컴포넌트의 소스 파일·라인으로 에디터가 바로 열립니다. Babel 설정도, 브라우저 확장도, React DevTools도 필요 없이 빌드 플러그인 하나만 추가하면 됩니다(Vite는 이것만으로 끝, Next·단독 번들러는 클라이언트에서 enableLocator() 한 줄 추가). v1은 소스를 다시 파싱했지만, v2는 컴파일러가 이미 계산해둔 위치를 가로챕니다 — 재파싱 경로가 아예 없습니다.

`npm v1.0.2` `번들러 6종 실측` `Turbopack · 서버 컴포넌트 지원` `Zero dependencies` `React 17-19 실측`

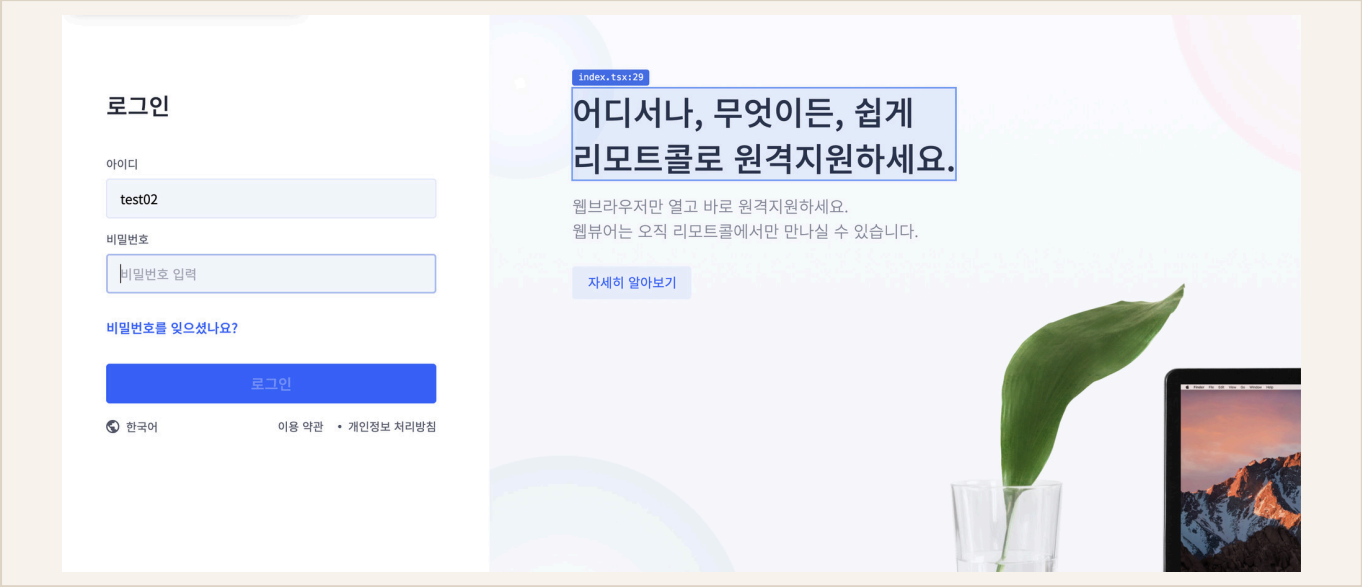
npm — https://www.npmjs.com/package/react-code-locator GitHub — https://github.com/dbckdgjs369/web-element-code-locator

---

왜 만들었나

화면에서 고칠 부분은 눈에 보이는데, 그게 어느 컴포넌트, 어느 파일에서 그려지는지 찾는 게 매번 번거로웠습니다. 규모가 큰 프로젝트나 남이 짠 코드에서는 눈에 보이는 텍스트를 복사해 전체 검색하고, 결과가 여러 개면 하나씩 열어보며 한참 헤맸습니다.

비슷한 도구가 없진 않았지만 대부분 React 내부의 fiber._debugSource에 의존했습니다. 그런데 React 19에서 이 필드가 제거되면서 그 방식은 더 이상 쓸 수 없게 됐습니다. React 버전에 묶이지 않는 방법이 필요했습니다.



Shift 호버 실제 서비스 화면에서 제목 요소에 커서를 올린 모습입니다.

어떻게 만들었나

1. 소스를 다시 파싱하지 않는다 — v2의 핵심 결정

문제 v1 아키텍처(npm 0.3.x)는 번들러의 transform 훅에서 @babel/parser로 소스를 재파싱해 위치를 주입했습니다. 동작은 했지만 모든 파일을 두 번 파싱하는 비용을 내고, 번들러마다 transform 훅 형태가 달라 어댑터가 무거웠습니다. 그런데 알고 보니 그 위치는 컴파일러가 dev 모드에서 이미 계산하고 있었습니다 — JSX를 jsxDEV() 호출로 바꾸며 5번째 인자로 파일·줄·칸을 넘기고, React는 받자마자 버립니다.

선택 재파싱 경로를 전부 삭제했습니다. 대신 react/jsx-dev-runtime import를 얇은 래퍼로 리다이렉트해, React가 버리기 직전의 __source를 WeakMap에 복사해 둡니다. 핵심 기능(JSX 호출 위치)은 이 훅 하나로 끝나고, 이 경로에서는 사용자 코드도 프롭도 건드리지 않습니다. React 19가 fiber._debugSource를 제거한 것과도 무관합니다 — 내부 필드가 아니라 모듈 경계에서 가로채기 때문입니다.

그런데 "컴파일러가 계산한 값이니 정확하다"는 전제가 한 환경에서 깨졌습니다. Vite의 @vitejs/plugin-react는 JSX 변환을 esbuild에 넘기는데, 그 전에 fast-refresh 프리앰블을 파일 앞에 붙입니다. esbuild는 이미 밀려난 텍스트를 기준으로 위치를 계산하므로, Vite에서 받는 줄 번호는 실측해보니 정확히 19줄씩 틀렸습니다. 전제의 정확한 조건은 "파싱과 JSX 변환 사이에 코드가 움직이지 않을 것"이었고, Next의 SWC는 한 패스라 이걸 만족합니다. 그래서 Vite에서만 이미 돌고 있는 Babel 패스에 얹혀(재파싱은 여전히 0회) AST 원본 좌표로 덮어씁니다. 두 값이 같은 WeakMap에 쓰이고 정확한 쪽이 바깥에서 실행되므로, 별도 분기 없이 정확한 값이 남습니다.

결과 파싱 비용 0, React 내부 API 의존 0. React 18·19는 환경별 테스트 레포로, 17은 실서비스 앱으로 실측했습니다.

2. 훅이 하나면 어댑터는 변주일 뿐이다 — Turbopack까지

문제 v1은 unplugin 어댑터 5종을 유지보수했고, 그래도 Turbopack은 불가능했습니다. transform 훅을 꽂을 공개 API가 Turbopack에 없었기 때문입니다. 오류도 경고도 없이 클릭만 조용히 안 먹혔습니다.

선택 v2의 훅은 "모듈 하나를 다른 모듈로 바꿔치기"라서, 각 번들러의 모듈 해석 지점만 찾으면 됩니다. Vite는 resolveId, rollup·esbuild도 각자의 resolve 훅, 그리고 Next.js는 turbopack.resolveAlias라는 공개 설정으로 같은 일을 해줍니다 — transform API는 없어도 alias는 있었습니다.

함정 단순 alias로는 순환 참조가 생깁니다. Vite의 resolveId는 importer가 누구인지 받아서 우리 래퍼 자신의 import만 예외로 둘 수 있지만, webpack·rspack·Turbopack·esbuild는 스펙 문자열만 보고 치환하기 때문에 래퍼가 부르는 react/jsx-dev-runtime까지 다시 래퍼로 바뀝니다. 그래서 설정 시점에 Node로 React의 실제 경로를 해석해, 절대 경로로 require하는 CJS 심(shim)을 생성했습니다 — alias가 자기 자신을 가리킬 수 없는 구조가 됩니다. 이 심에는 projectRoot도 빌드 타임에 구워 넣습니다. SWC는 __source에 절대 경로를 박고 브라우저는 프로젝트 위치를 알 방법이 없어서, 이게 없으면 "Copy path" 결과가 빌드한 기계에서만 유효해집니다. 레지스트리 자체는 globalThis의 심볼 키에 매달립니다 — pnpm이나 모노레포에서 패키지 복사본이 둘 생겨도 하나의 WeakMap으로 수렴해야 하기 때문입니다.

Turbopack 공개 설정은 있었지만 제약이 세 겹이었습니다. Turbopack은 절대 경로를 거부하고(alias 값도, 심 내부 require()도 — "server relative imports are not implemented yet"), 프로젝트 루트 밖 참조도 거부해서 래퍼를 심 옆으로 복사해야 했습니다. 결정적으로 Next는 next/dist/compiled/react를 벤더링하기 때문에, 평범한 react를 감싸면 React 내부 상태가 두 인스턴스로 갈려 첫 렌더에서 앱이 죽습니다. webpack 모드에도 별개 함정이 있었습니다 — App Router는 top-level resolve.alias가 아니라 module rule 레이어별 alias로 React를 해석하고 그 rule이 top-level을 이겨서, module.rules를 재귀로 훑어 각 rule에도 같은 alias를 넣어야 했습니다.

결과 v1의 최대 한계였던 Turbopack이 지원 목록에 들어왔습니다. react-code-locator/next의 withReactCodeLocator() 래퍼 하나로 next dev와 next dev --turbopack 양쪽에 동시에 걸리고, 둘 다 Chromium E2E로 실측했습니다.

3. 서버 컴포넌트 — 위치가 서버에만 있는 문제

문제 App Router의 기본값인 서버 컴포넌트는 jsxDEV가 Node에서 실행됩니다. 위치는 서버 쪽 WeakMap에 쌓이고, 브라우저로 오는 직렬화된 페이로드에는 __source가 없습니다. 이대로면 App Router 화면 대부분이 조회 불가입니다.

선택 React 19가 RSC 엘리먼트마다 owner stack을 실어 보내고 클라이언트가 fiber._debugStack에 남겨두는 걸 이용했습니다. 다만 좌표가 컴파일된 서버 모듈 기준이라, Next가 자기 에러 오버레이용으로 열어둔 소스맵 서비스(/__nextjs_original-stack-frames)에 동일 출처 POST 한 번으로 원본 좌표로 되돌립니다. 이 경로는 비동기라 locateAsync()로 분리하고, 한 번 받은 답은 메모이즈합니다. 첫 왕복이 끝나기 전에는 하이라이트를 지우는 대신 직전 상태를 유지해 깜빡임을 없애고, 답이 왔을 때 커서가 이미 다른 요소로 떠났으면 그리지 않습니다.

결과 서버 컴포넌트도 webpack·Turbopack 양쪽에서 원본 app/page.tsx 좌표로 점프합니다. 크롬 익스텐션도 CDP도 없이요.

4. 프로덕션 번들에 영향을 주지 않으려면

문제 디버깅 편의를 위해 넣은 도구가 사용자에게 나가는 번들을 키우면 쓸 이유가 없습니다.

선택 가로채는 대상이 react/jsx-dev-runtime — 이름 그대로 dev 빌드에서만 import되는 모듈입니다. 프로덕션 빌드는 jsx-runtime을 쓰므로 훅이 걸릴 자리 자체가 없습니다. 외부 의존성도 0개입니다.

결과 프로덕션 번들 0바이트 — 노력해서 지운 게 아니라, 프로덕션에는 훅이 걸릴 자리가 애초에 없습니다. Vite 플러그인은 apply: "serve"로 dev 서버에만 붙고, webpack·Next 통합은 프로덕션 모드에서 스스로 비활성됩니다.

5. styled 컴포넌트가 전부 styled.tsx로 빠지는 문제

문제 실무 코드베이스에 붙여보니 거의 모든 클릭이 styled.tsx로 떨어졌습니다. styled.h1으로 만든 컴포넌트는 태그드 템플릿이라 그 안에 JSX가 없습니다. 그래서 소스 위치를 못 찾고 정의된 자리로 되돌아갔습니다. 정작 보고 싶은 건 그걸 갖다 쓴 화면인데, 스타일 정의 파일만 열리니 쓸모가 없었습니다.

선택 위로 훑어 올라가면서 프로젝트 안에 있는 가장 가까운 JSX 위치를 먼저 찾도록 순서를 바꿨습니다. 컴포넌트 정의 위치는 그마저 없을 때의 최후 수단으로 내렸습니다.

결과 위 스크린샷이 그 결과입니다. WelcomeTitle은 styled.tsx에 정의돼 있지만 라벨은 그걸 쓴 index.tsx:29를 가리킵니다.

6. 편의 기능이 앱을 죽이면 안 된다

문제 소스 위치를 컴포넌트에 속성으로 대입하다가, 얼려둔(frozen) 컴포넌트에 붙이려는 순간 앱 전체가 죽은 적이 있습니다. ESM은 strict mode라 실패한 대입이 그냥 넘어가지 않고 예외를 던집니다. 디자인 시스템 export나 SVG 변환 결과물이 이런 경우입니다.

선택 두 겹으로 접근했습니다. 핵심 경로(JSX 호출 위치)는 대입 자체가 없는 구조로 — 위치는 엘리먼트의 props 객체를 키로 하는 WeakMap에만 있어 frozen이든 sealed든 관계없습니다. 여전히 속성 대입이 필요한 부가 경로(Vite 한정 구현 모드)는 Object.isExtensible 가드로 감싸 붙일 수 없으면 조용히 건너뜁니다 — 그 컴포넌트만 구현 모드가 빠지고 나머지는 전부 동작합니다. 이 가드 덕분에 판단 방향도 뒤집을 수 있었습니다. 처음에는 HOC 이름 allow-list로 컴포넌트를 골라냈는데 프로젝트 로컬 HOC를 전부 놓쳤고, 그래서 모든 호출 표현식을 후보로 받고 위험은 가드가 흡수하도록 바꿨습니다. 반환값이 객체·함수가 아닌 경우까지 같은 자리에서 걸러집니다.

결과 실서비스 앱(React 17 + webpack 5 + styled-components)에 붙여 페이지 에러 0건으로 실측했습니다. 소스로 점프하는 건 편의 기능입니다. 그게 앱을 죽이는 건 어떤 이유로도 정당화되지 않습니다.

7. 실서비스 앱에 붙이자 번들러가 세 번 막았다

문제 playground에서 전부 통과한 패키지를 실서비스 CRA(eject) 앱에 붙였더니 빌드부터 서지 않았습니다. ① CRA의 ModuleScopePlugin이 src/ 밖을 가리키는 import를 차단해 "falls outside of the project src/ directory" 에러가 86건 ② 심을 .cjs로 내보냈더니 CRA의 catch-all file-loader가 그걸 정적 에셋으로 번들해서, 모든 import { jsxDEV }가 URL 문자열에 바인딩되며 undefined가 됐습니다 ③ 심을 module.exports = wrap(...) 형태로 쓰니 webpack의 CJS 파서가 named export를 추적하지 못해 "only default export is available" 경고가 JSX 파일마다 3,025번 찍혔습니다.

선택 세 개 다 사용자가 설정을 고치지 않는 방향으로 해결했습니다. ①은 플러그인 클래스 이름에 의존하지 않고 구조로 감지해 allow-list에 심과 래퍼를 추가 ②는 확장자를 .js로 바꾸고 심 디렉터리에 {"type":"commonjs"}를 함께 써서 상위 "type": "module"에 오염되지 않게 ③은 정적 exports.x = 형태로 바꿔 파서가 추적할 수 있게 만들었습니다.

결과 테스트 레포에서는 하나도 재현되지 않던 문제들입니다. 실서비스에 붙여보지 않았다면 "지원한다"고 적어둔 채로 남았을 것들이라, 이후 지원 매트릭스에 실서비스 검증 칸을 따로 두었습니다.

8. 조용히 실패하지 않는다

문제 메뉴에서 "Open in editor"를 눌렀는데 아무 일도 일어나지 않는 제보를 받았습니다. 엔드포인트는 200을 돌려주고 있었습니다. 원인은 dev 서버 프로세스의 $EDITOR가 vi였던 것 — 터미널 에디터를 detached로 띄우면 붙을 TTY가 없어 뜨지도, 죽지도 않고 사라집니다.

선택 에디터 결정을 4단계 폴백으로 바꿨습니다. REACT_CODE_LOCATOR_EDITOR → $VISUAL/$EDITOR(단 vi·vim·nano 같은 터미널 에디터는 의도적으로 건너뜀) → 실행 중인 GUI 에디터 자동 탐지(프로세스 목록에서 VS Code 계열·JetBrains 전 제품군·Sublime·Zed를 찾습니다) → code. 그리고 고른 결과를 서버 콘솔에 한 줄 남깁니다.

원칙 이건 이 프로젝트에서 반복해서 물린 지점입니다. v1의 최악의 버그도 삼켜진 404였고, 겉으로는 무동작과 구별되지 않았습니다. 그래서 실패하는 자리마다 말을 하게 만들었습니다 — 엔드포인트가 실패하면 상태 코드·감지한 dev 서버·URL을 함께 보고하고, 에디터 실행 파일이 없으면(ENOENT는 예외가 아니라 비동기 이벤트라 리스너가 없으면 출력조차 없습니다) 해결 방법까지 붙여 알리고, 심 해석이 실패하면 빌드를 세우는 대신 경고 후 기능만 내려놓습니다.

결과 VS Code를 쓰지 않는 환경에서도 설정 없이 열리고, 안 열릴 때는 왜 안 열렸는지가 콘솔에 남습니다. 1.0.2로 배포했습니다.

결과

- 재파싱 아키텍처(npm 0.3.x)를 버리고 jsx-dev-runtime 가로채기 단일 훅으로 전면 재설계 — 1.0.0으로 메이저 승격 후 1.0.2 배포.
- 훅 하나를 각 번들러의 모듈 해석 지점에 꽂는 방식으로 Vite·webpack·rspack·rollup·esbuild + Next.js(webpack·Turbopack) 지원 — v1에서 불가능하던 Turbopack과 서버 컴포넌트가 들어왔습니다.
- 지원 매트릭스 전 칸을 실측 — Vite·Next(webpack·Turbopack·서버 컴포넌트)는 Chromium 실제 렌더 E2E로, webpack·rspack·rollup·esbuild 단독은 각 번들러 산출물을 실행해 좌표를 대조하는 방식으로. 추론으로 적은 칸이 없습니다.
- 실서비스 앱(React 17 + webpack 5 + styled-components)에 붙여 탐지 정확도와 에디터 점프까지 검증 — 테스트 레포에서 재현되지 않던 번들러 함정 3종을 여기서 잡았습니다.
- 동작하지 않는 조건(classic + React 19)을 숨기지 않고 문서에 먼저 밝혔습니다.

P3 · VSCode Extension + Node.js 릴레이 서버 · 2025.12 — 2026.09 · v2.1.6

Custom Tunnel

로컬 개발 서버를 공개 URL로 즉시 열어, 모바일 등 외부 기기에서 빌드·배포 없이 확인하는 VSCode 확장입니다. 터널 생성부터 QR 접속, 모바일 브라우저의 콘솔 로그 확인까지 에디터 사이드바 안에서 끝납니다.

빌드·배포 120초 → 0 | 실무 적용 | VSCode 확장 v2.1.6

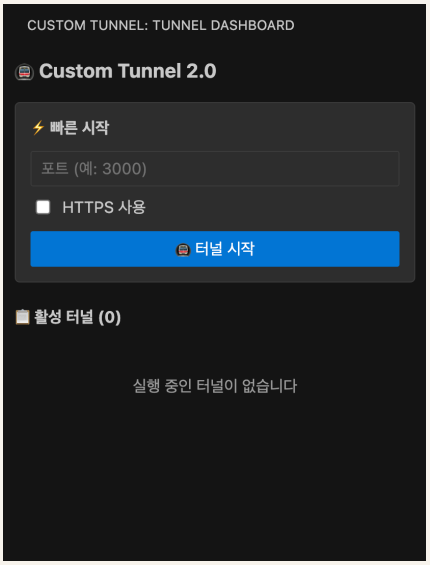
GitHub — https://github.com/dbckdgjs369/debug_tool

왜 만들었나

로컬에서 만든 화면을 실제 폰에서 확인하고 싶은데, 그때마다 빌드하고 배포 서버에 올려(약 120초) 폰으로 접속해야 했습니다. 여백 몇 픽셀, 터치 영역 하나 확인하려고 2분씩 기다리는 게 반복되니 개발 흐름이 자꾸 끊겼습니다.

더 곤란한 건 모바일 브라우저에는 콘솔이 없다는 점이었습니다. 폰에서만 재현되는 버그를 만나면 원인을 볼 방법이 없어, alert()를 심고 다시 2분을 기다리는 식으로 디버깅했습니다.

기존 터널링 서비스도 있었지만, 매번 별도 CLI를 띄우고 URL을 복사해 폰에 옮기는 과정이 번거로웠고 원격 콘솔은 어차피 해결되지 않았습니다. 에디터를 벗어나지 않고, 콘솔까지 같이 보이는 도구가 필요했습니다.



대시보드 초기 화면 포트 입력 → HTTPS 여부 체크 → 터널 시작, 세 단계가 전부입니다.

어떻게 만들었나

1. 방화벽 뒤의 로컬 서버를 어떻게 외부에 노출하나

문제 — 공개 인터넷에서 내 노트북의 localhost:3000으로 직접 들어올 방법이 없습니다. 공유기·방화벽이 인바운드 연결을 막고, 포트포워딩은 매번 설정할 것도 많고 보안상 내키지 않았습니다.

선택 — WebSocket 기반 리버스 프록시로 방향을 뒤집었습니다. 로컬 클라이언트가 바깥으로 나가서 릴레이 서버에 WS 연결을 걸어두면, 서버는 외부 요청이 들어올 때 그 연결에 요청을 실어 보냅니다. 아웃바운드 연결은 방화벽이 막지 않으므로 설정이 필요 없습니다.

결과 — 사용자는 포트 번호만 입력하면 됩니다. 네트워크 설정은 건드릴 게 없습니다.

2. 콘솔이 없는 모바일 브라우저를 어떻게 디버깅하나

문제 폰에서만 재현되는 버그가 있어도 console.log를 볼 수 없습니다. USB 원격 디버깅은 케이블·드라이버·기기별 설정이 제각각이라 매번 손이 많이 갔습니다.

선택 터널을 지나는 HTML 응답에 콘솔 캡처 스크립트를 주입했습니다. 주입된 스크립트가 console 메서드를 감싸 호출을 가로채고, 같은 WS 채널로 릴레이 서버에 흘려보내면 확장이 받아 사이드바에 그립니다. 앱 코드는 한 줄도 고치지 않습니다.

결과 케이블 없이, 폰에서 뜬 로그가 에디터 사이드바에 실시간으로 쌓입니다. 레벨·키워드 필터까지 붙였습니다.

3. 응답이 이미지·폰트면 어떻게 되나

문제 터널은 WebSocket 메시지로 응답을 실어 나릅니다. HTML·JS·CSS는 문자열로 다뤄도 되지만, 이미지·폰트 같은 바이너리를 문자열로 취급하면 내용이 손상됩니다. Content-Type도 함께 보존해야 브라우저가 제대로 해석합니다.

선택 처음에는 Content-Type을 보고 텍스트와 바이너리를 갈라, 바이너리와 SVG를 Base64로 인코딩해 중계했습니다. 문자열 채널 하나로 모든 응답을 실을 수 있어 구현이 단순했습니다.

재측정 동작은 했지만 Base64는 3바이트를 4글자로 바꾸는 인코딩이라 전송량이 원본의 4/3입니다. 터널은 로컬 → Render → 브라우저를 왕복하는 경로여서 이 33% 가 그대로 지연이 됩니다. 이미지·폰트가 많은 페이지에서 더 커집니다.

교체 WebSocket 은 원래 바이너리 프레임을 보낼 수 있으니, 응답 본문을 인코딩 없이 그대로 실어 보내도록 바꿨습니다. 대신 프레임 앞에 종류·식별자를 붙였습니다 — [1B 종류][4B id][페이로드]. HTTP 응답 본문과 WS 메시지가 같은 채널을 공유하므로 구분이 필요했고, 이게 프로토콜 v3 입니다.

결과 이미지·폰트·아이콘이 포함된 실제 페이지가 그대로 렌더링되고, Base64 오버헤드 33%가 사라졌습니다. 반대 방향(브라우저 → 로컬)의 요청 본문은 아직 Base64 입니다 — 업로드가 아니면 대개 작아서 우선순위를 뒤로 뒀습니다.

4. 꺼낸 화면이 왜 반쯤만 동작하나

문제 화면은 제대로 뜨는데 일부 기능이 조용히 죽어 있었습니다. 에러도 안 나고 요청·응답도 멀쩡해서 중계 코드만 한참 들여다봤습니다.

추적 브라우저 네트워크를 보니 wss://<터널주소>:3001/ws 처럼 터널 밖으로 나가는 요청이 있었습니다. 원인은 제 코드가 아니라 번들 안이었습니다 — dev 서버가 클라이언트 번들에 자기 주소를 ?hostname=0.0.0.0&port=3001 같은 문자열로 구워 넣습니다. 브라우저는 페이지를 어디서 받았는지와 무관하게 그 주소로 다이얼하니, 로컬 주소가 박힌 통신만 골라서 새어 나갔습니다. 페이지를 꺼내는 것과 꺼낸 페이지가 온전히 동작하는 건 다른 문제였습니다.

선택 번들을 고치는 건 사용자 프로젝트를 건드리는 일이라 하지 않았습니다. 대신 이미 주입하고 있던 스크립트에서 WebSocket·EventSource·fetch·XMLHttpRequest 네 개를 감싸, 인자로 들어온 주소가 로컬 절대 주소면 현재 페이지 origin으로 바꿔 넘겼습니다. 브라우저가 실제로 다이얼하는 마지막 지점에서 교정하는 방식이라, 어떤 빌드 툴이 어떤 주소를 박아 넣든 같은 곳에서 잡힙니다.

결과 꺼낸 화면이 로컬에서 보던 것과 똑같이 동작합니다. 앱 코드도, 빌드 설정도 손대지 않습니다 — 유일한 예외는 Next.js dev 서버의 origin 허용 설정 한 줄이고, 이것도 "첫 예외"로 문서에 밝혀 뒀습니다. 덤으로 저장하면 폰이 새로고침 없이 바뀌는 것도 여기서 같이 따라왔습니다. v2.1.3입니다.

5. 앱이 쓰는 실시간 채널은 어떻게 통과시키나

문제 주소를 교정해도 채널 자체를 나를 수 없으면 여전히 반쪽입니다. HTTP 요청·응답만 중계하면 요청 하나에 응답 하나로 끝나는 트래픽은 통과하지만, 앱이 직접 여는 WebSocket과 SSE는 성격이 다릅니다 — 응답이 끝나지 않는 스트림이고, 연결이 양방향으로 살아 있어야 합니다.

선택 둘을 따로 다뤘습니다. 앱 WebSocket은 릴레이에서 별도 연결로 승격시키고, 서브프로토콜 협상과 양방향 종료(어느 쪽이 닫아도 반대쪽에 닫힘이 전달되도록 — 단, 무료 호스팅 프록시 경유 시 닫힘 코드가 1000/1006으로 뭉개지는 건 알려진 제약으로 문서화)까지 옮겼습니다. 핸드셰이크 결과가 다르면 앱이 프로토콜을 잘못 고릅니다. SSE는 응답을 모아서 보내면 안 되므로 버퍼링 없이 도착한 청크를 그대로 흘려보냈습니다.

결과 채팅·알림처럼 실시간 통신을 쓰는 화면도 터널로 그대로 확인됩니다. 4·5번을 지나서야 "로컬을 꺼내면 그냥 동작한다"고 말할 수 있게 됐습니다. 이 단계에서 릴레이는 HTTP 프록시가 아니라 다중 채널을 나르는 전송 계층이 됐고, 그래서 프레임에 종류·식별자를 붙이는 프로토콜 v3 이 필요해졌습니다.

결과

- 실무에 직접 적용해 빌드·배포 120초 과정을 제거. 포트 번호만 넣으면 지금 이 순간의 로컬 화면이 폰에 뜨고, 로컬에서 보던 것과 똑같이 동작합니다 — 실시간 통신도, 저장 즉시 반영도 그대로입니다.
- 모바일 전용 버그를 alert() 없이 로그를 보면서 잡게 됐습니다.
- VSCode 확장 v2.1.5까지 vsix 패키징·배포, v2.1.6 개발 중. 2.0에서 사이드바 대시보드 UI로 전면 개편했습니다.
- 프로토콜 버전 협상·주입 스크립트·앱 WS 중계·고아 프로세스 정리를 검증 61개로 묶어, 프로토콜을 바꿀 때 무엇이 깨지는지 먼저 알고 손댈 수 있게 했습니다.
- 터널을 지나면 로컬 직결보다 느립니다. 재보고 용도를 정했습니다 — 레이아웃·동작·모바일 버그를 확인하는 도구고, 성능을 재는 자리에는 쓰지 않습니다.

P4 · Electron 데스크탑 앱 (macOS · Windows) · 2026.08 — 진행중 · v0.1.0

CC Talk

여러 Claude Code 세션을 PC 카카오톡처럼 관리하는 데스크탑 앱입니다. 세션 하나 = 채팅방 하나 = 창 하나로, 여러 프로젝트를 동시에 굴리면서 어느 방이 내 대답을 기다리는지 한눈에 봅니다.

매일 쓰는 도구 | TUI 스크래핑 → 헤드리스 stream-json | 캐시 읽기 78% 실측 후 압축 정책 재설계 | macOS · Windows 동시 지원

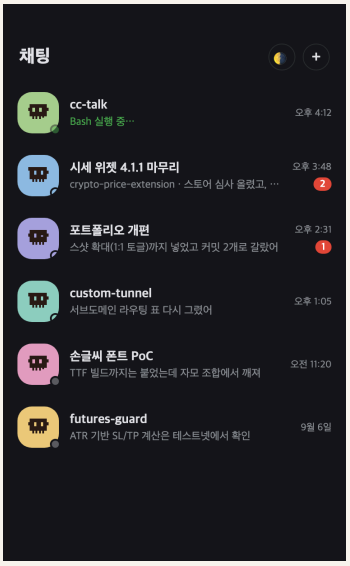
GitHub — https://github.com/dbckdgjs369/claude-talk

왜 만들었나

Claude Code 를 하루 종일 쓰면서 터미널 창이 계속 늘어났습니다. 프로젝트마다 세션을 띄우고, 한 프로젝트 안에서도 작업을 나눠 여러 개를 병렬로 굴리게 됩니다. 그런데 창은 자기가 무슨 작업을 하고 있는지 밖에서 알려주지 않습니다. 창을 하나씩 눌러 스크롤을 올려보며 「이게 무슨 작업이었지」를 매번 다시 찾는 게 일이 됐습니다.

더 컸던 문제는 끝나도 아무 말이 없다는 점이었습니다. 다 끝난 창은 그냥 조용히 멈춰 있으니, 다른 걸 하는 사이에 그 세션을 잊습니다. 한참 뒤에 돌아와서는 내가 뭘 시켰는지부터 다시 읽어야 하고, 그 사이 다른 창을 확인하러 오가느라 하던 일의 맥락도 끊깁니다. 세션이 늘어날수록 기다리는 시간이 아니라 다시 찾는 시간이 늘었습니다.

그런데 이 상황은 이미 익숙한 모양이 있습니다 — 여러 사람과 동시에 대화하는 메신저입니다. 방마다 이름이 붙어 열어보지 않아도 누구와의 대화인지 알고, 안 읽은 방에는 뱃지가 붙고, 상대가 말을 하면 알림이 옵니다. 세션을 사람처럼 취급하면 「어느 창이 무슨 작업인지」와 「끝났는지」가 이미 풀린 UI 로 그대로 해결됩니다.



방 목록 — 카톡 메인 그대로 방마다 색과 아바타가 달라 줄만 보고 어느 프로젝트인지 압니다. 상태 점이 초록이면 지금 그 세션이 돌고 있고, 뱃지 숫자는 내가 안 본 답변 수입니다. 창을 하나씩 눌러 스크롤을 올려보던 「이게 무슨 작업이었지」가 이 한 화면으로 대체됐습니다.

어떻게 만들었나

1. 터미널 화면을 읽어서 만들었더니 전부 어긋났다

문제 첫 버전은 PTY 로 Claude Code 를 띄우고 TUI 화면을 읽는 구조였습니다. 사람이 보는 화면을 기계가 읽으니 되는 것처럼 보였지만, 실제로는 계속 깨졌습니다 — 「Quick safety check」 신뢰 확인 창이 입력을 삼켜버리고, TUI 가 입력을 받을 준비가 됐는지 알 방법이 없어 화면 모양으로 추측했고, 그 추측이 틀릴 때를 대비해 8초 뒤 재전송하는 안전망까지 달았습니다. 대화 기록 파일이 게으르게 생성돼 서로 기다리다 멈추는 일도 있었습니다.

선택 고치는 대신 전제를 버렸습니다. Claude Code 에는 사람용 TUI 말고 기계용 입출력이 있습니다 — -p --input-format stream-json --output-format stream-json. 이걸 자식 프로세스로 띄우고 구조화 JSON 이벤트로 주고받도록 다시 세웠습니다. 추측하던 것이 명시적 신호가 됐습니다 — 「준비됐나」를 화면 모양으로 재던 것이 init 이벤트로, 「턴이 끝났나」가 result 이벤트로 바뀌었습니다. 재전송 안전망·준비 감지 휴리스틱·신뢰 다이얼로그 회피가 전부 필요 없어졌습니다 없어진 게 버그가 아니라 버그의 종류였습니다. 「화면을 잘못 읽는다」는 클래스가 통째로 사라졌습니다. 개별 증상을 하나씩 잡는 것과는 다릅니다 대신 사람용 편의는 직접 만들어야 했습니다 — 슬래시 커맨드 자동완성, 정지 버튼, 권한 모드 전환은 TUI 가 해주던 것입니다. 다만 이건 만들면 되는 일이고, 화면을 읽는 건 아무리 잘 만들어도 어긋납니다

결과 v1 에서 필요했던 방어 코드가 한 줄도 남지 않았습니다. 소스에 「TUI 를 기계로 조종할 생각은 다시 하지 말 것」을 남겨, 나중에 편해 보여서 되돌리는 일이 없게 했습니다.

2. 내가 만든 앱이 하루 비용의 78%를 캐시 읽기로 태우고 있었다

문제 헤드리스 모드에는 TUI 의 자동 압축이 없습니다. 그래서 앱이 직접 걸었는데, 임계를 모델 한도의 80%로 뒀습니다(1M 모델이면 80만). 그럴듯해 보였지만 에이전트 루프의 비용 구조를 잘못 봤습니다 — 도구 호출 하나가 API 요청 한 건이고, 매 요청이 컨텍스트 전량을 다시 읽습니다. 실측하니 요청당 평균 컨텍스트가 75.7만·89.1만(8/29·8/30)까지 올라가 있었고, 캐시 읽기가 하루 비용의 78%였습니다. 압축 기록을 다 뽑아보니 CLI 시절 자동 압축은 1~3만에서 돌던 것이, 앱을 쓰기 시작한 뒤 전부 80만~100만이었습니다.

선택 임계를 모델 한도와 무관한 절대값 180k 로 내렸습니다. 그러다 같이 드러난 버그가 하나 더 있었습니다 — 모델 한도를 재는 함수가 opus·sonnet 을 전부 1M 으로 보고 있어서, 실제로는 200k 인 일반 모델에서 압축이 걸리기도 전에 「Prompt is too long」으로 방이 죽었습니다. [1m] 변형만 1M 으로 한정했습니다. 그런데 이 수정을 나중에 다시 뒤집었습니다. 압축 기록 37건을 재보니 압축 한 번에 걸리는 시간의 중앙값이 171초인데 크기와 거의 무관했습니다(163k → 238초, 807k → 191초). 한 번 값이 고정이면 횟수를 줄이는 게 그대로 이득입니다. 게다가 180k 라는 숫자 자체가 「TUI 는 한도의 92% 에서 압축한다」는 근거 없는 추측에서 나온 값이었는데, 실제 기록을 보니 자동 압축이 걸린 지점은 사실상 한도였습니다. 정답을 하나로 정하지 않았습니다 — 비용은 「요청 수 × 평균 컨텍스트」고 평균은 대략 (임계+바닥)/2 입니다. 임계를 올리면 압축 횟수는 줄지만 매 요청이 비싸집니다. 짧게 묻고 끝나는 방과 며칠씩 파고드는 방의 답이 같을 수 없어서 임계를 방마다 고르는 값으로 뒀습니다 한도를 꽉 채우게 두지는 않았습니다 — 다 차면 /compact 자체가 「Prompt is too long」으로 실패해 방이 통째로 막힙니다. 압축도 턴이라 들어갈 자리가 있어야 해서 8% 를 남깁니다 잠든 방을 깨우는 값은 임계와 분리했습니다 — 방을 재우기 전에 압축해두면, 임계를 얼마로 두든 깨우는 값은 바닥값입니다

결과 「느낌상 비싸다」가 아니라 요청당 평균 컨텍스트·압축 소요시간·캐시 읽기 비중이라는 숫자로 판단하게 됐습니다. 그 숫자가 제 첫 수정을 뒤집게 만들었다는 게 이 블록의 핵심입니다 — 180k 는 추측이었고, 뒤집은 근거는 실측 37건입니다. 두 선택의 근거를 코드 주석에 그대로 남겨, 다음에 이 값을 만지는 사람이 같은 추측을 반복하지 않게 했습니다.

3. 한글 폴더에서 만든 방만 기록이 복원되지 않았다

문제 앱을 다시 켜면 대화 기록을 ~/.claude/projects/<인코딩된 폴더 경로>/ 에서 읽어옵니다. 그런데 한글 폴더명인 방만 기록이 비어 있었습니다. 영문 폴더는 전부 정상이었습니다.

선택 원인은 같은 글자를 저장하는 두 가지 방식이었습니다. macOS 파일시스템이 넘겨주는 경로는 NFD(「한」을 ㅎ+ㅏ+ㄴ 으로 분해)인데, Claude Code 가 폴더명을 만들 때 쓰는 기준은 NFC(「한」을 한 글자로)입니다. 눈에 보이는 문자열은 완전히 같고 바이트만 다릅니다. 경로 인코딩 함수에서 normalize('NFC') 를 먼저 걸었습니다. 기록 포맷을 아는 코드는 파일 하나에만 두었습니다 — jsonl 을 읽고 쓰는 지식이 여기저기 흩어져 있으면, Claude Code 가 포맷을 바꿀 때 고칠 곳을 찾아다녀야 합니다. 지금은 한 파일만 봅니다

결과 한글 폴더에서도 기록이 그대로 복원됩니다. 「영문에서는 되는데」가 곧 유니코드 정규화 문제라는 걸 알게 된 뒤로는, 경로를 문자열로 다루는 곳마다 정규화를 먼저 확인합니다.

4. 앱에서 한 대화가 터미널 목록에서 사라졌다

문제 앱과 터미널은 같은 기록 파일을 봅니다. 그래서 앱에서 하던 대화를 터미널에서 claude --resume 으로 이어갈 수 있어야 하는데, 목록에 안 떴습니다. 헤드리스로 만든 세션은 파일에 「SDK 가 만든 것」으로 표시되고, 터미널 픽커가 그걸 사람이 볼 필요 없는 세션으로 보고 숨깁니다.

선택 세션이 끝날 때와 앱을 켤 때 그 표시를 일반 CLI 세션으로 바꿔 씁니다. 원칙을 하나 정하고 거기서 나머지를 끌어냈습니다 — 「앱과 Claude Code 는 같은 데이터를 본다」. 기록 파일 하나 = 방 하나. 아무것도 합치지 않습니다 — 같은 폴더의 세션을 묶어 보여주는 게 「깔끔」해 보이지만, 그러면 터미널 픽커와 개수가 안 맞습니다. 한 프로젝트에서 세션 여러 개를 병렬로 굴리는 게 정상 패턴이라 묶을 이유도 없었습니다 방 이름도 같은 방식으로 — 제목은 Claude Code 가 첫 턴 뒤에 기록에 남기는 자동 제목을 그대로 씁니다. 앱에서 이름을 바꿀 때도 별도 저장소가 아니라 기록 파일에 씁니다. 그래야 터미널 픽커에도 바뀐 이름이 보입니다 방을 지우면 기록 파일까지 지웁니다 — 앱에서만 숨기면 터미널 목록에는 남아 두 곳이 어긋납니다. 이어가기로 생긴 사본까지 따라가 지웁니다

결과 앱과 터미널을 섞어 써도 목록이 어긋나지 않습니다. 앱에서 시작한 대화를 터미널에서 잇고, 터미널에서 한 대화가 앱 목록에 뜹니다. 「앱 안에서만 맞는 상태」를 만들지 않은 게 이 기능의 전부입니다.

5. 아무것도 안 하는데 노트북이 뜨거웠다

문제 방을 열어두면 가만히 있어도 팬이 돌았습니다. 재보니 렌더러가 상시 약 10% CPU 를 먹고, 자식 프로세스들이 1.15GB 를 잡고 있었습니다.

선택 범인은 「생각 중」 옆의 점 세 개 애니메이션이었습니다. content 를 . → .. → ... 로 바꾸는 방식이었는데, content 변경은 레이아웃을 다시 돌립니다. 그 재계산이 메시지 1,000개짜리 문서 전체로 번져 초당 3~4회 전체 재배치가 일어났습니다. 같은 모양을 레이아웃 0회로 — 점 세 개를 미리 그려두고 opacity 만 흔듭니다. 그리고 contain: layout style 로 이 요소의 재계산이 문서 밖으로 번지지 않게 격리했습니다 안 보이는 창을 위해 일하지 않습니다 — 파일 변경 감지(4초)·새 세션 스캔(20초)·사용량 조회(10분)를 보이는 창이 없으면 건너뜁니다. 특히 사용량 조회는 매번 무거운 프로세스를 통째로 띄우던 것이었습니다 창을 닫아도 프로세스는 2시간 살려 둡니다 — 처음엔 5분 뒤에 내렸는데, 이어가기(resume)가 대화 본문을 캐시에서 못 읽고 통째로 다시 쓰는 걸 실측(재작성 216회, 전체 입력 비용의 25%)하고 유예를 2시간으로 뒤집었습니다. 메모리는 대신 동시 유지 세션 4개 상한으로 묶고, 턴이 도는 중이면 미룹니다 접근성 설정을 켠 사용자에겐 점을 고정합니다 — 애니메이션을 줄이라는 설정이 켜져 있으면 흔들지 않습니다

결과 가만히 둔 방이 CPU 를 먹지 않습니다. 원인이 「Electron 이 무겁다」가 아니라 CSS 속성 하나의 선택이었다는 게 이 문제의 교훈입니다 — 같은 그림을 content 로 그리면 레이아웃, opacity 로 그리면 합성입니다.

6. 턴이 끝나야 말이 들어가는 줄 알았다

문제 세션이 도는 중에 사용자가 말을 보내면 무조건 큐에 넣고 턴이 끝날 때까지 기다렸습니다. 「엉뚱한 방향으로 가고 있는데」 하고 끼어들어도, 그 말이 닿는 건 잘못된 작업이 다 끝난 뒤였습니다.

선택 가정을 실험으로 확인했습니다. 대기를 6번 반복하는 턴을 만들고 두 번째 반복 직후에 말을 넣어봤더니, 세션이 남은 4번을 버리고 그 말에 답했습니다. 헤드리스 stream-json 도 도구 호출 경계에서 입력을 집어갑니다. 큐에 넣을 이유가 없었습니다. 모든 턴이 그렇지는 않습니다 — 도구를 안 쓰는 순수 텍스트 생성 턴은 경계가 없어 끝난 뒤에 처리됩니다. 터미널도 같습니다. 그래서 「즉시 전달」이 아니라 「즉시 전달하고, 경계가 올 때까지는 대기로 보인다」가 정확한 서술입니다 내가 보내지 않은 턴도 있습니다 — 터미널에서 이어가면 앱이 시작하지 않은 턴이 돕니다. 이걸 감지해 상태를 되돌리지 않으면 실제로 도는 중에 「대기 중」으로 보입니다 압축 턴에는 끼어들지 않습니다 — 압축 중에 사용자 말이 섞이면 압축 결과가 오염됩니다

결과 방향이 틀렸을 때 그 자리에서 돌릴 수 있게 됐습니다. 「큐에 넣어야 안전하다」는 건 확인해본 적 없는 가정이었고, 재보니 사실이 아니었습니다.

7. 두 방이 사람 없이 무한히 주고받으면 어떻게 되나

문제 방끼리 @호출명 으로 말을 전달하게 만들자마자 위험이 하나 생겼습니다 — A 가 B 를 부르고 B 가 A 를 부르면 사람 없이 둘이 계속 주고받습니다. 그동안 토큰은 계속 나갑니다. 게다가 이건 실행 중에 눈으로 잡기 어렵습니다 — 두 방을 동시에 보고 있지 않으면 모릅니다.

선택 홉(전달 횟수) 상한을 두고, 그 판단 로직을 Electron 에서 떼어낸 별도 파일로 옮겼습니다. 이유는 하나입니다 — 앱을 띄우지 않고 검증할 수 있어야 해서입니다. 앱을 켜서 두 방을 만들고 무한 루프를 재현하는 방식으로는 이걸 안심하고 고칠 수 없습니다. 압축 요약에 섞인 @ 는 전달하지 않습니다 — 압축된 요약문에 예전 호출명이 남아 있으면 아무도 하지 않은 말이 전달됩니다 상한에 걸린 전달은 조용히 버리지 않습니다 — 보낸 방에 회색 안내를 한 줄 남깁니다. 조용히 막으면 「왜 전달이 안 됐나」를 알 수 없습니다

결과 방끼리 협업하게 두면서도 비용이 새는 경로를 막았습니다. 「위험한 로직은 앱 없이 검증되는 자리에 둔다」가 이 파일을 떼어낸 이유고, 파일 맨 위에 그 이유를 적어뒀습니다.

8. 같은 코드가 Windows 에서는 닫기 버튼이 없었다

문제 macOS 에서 맞춰둔 창 설정을 Windows 에서 그대로 쓰니 닫기 버튼이 통째로 사라졌습니다. mac 의 「제목줄 숨기고 신호등만 남기는」 설정이 Windows 에서는 무시되는데, 무시되면서 네이티브 버튼도 안 그려집니다. 그 밖에도 $HOME 이 없고, 셸이 다르고, 알림이 안 뜨고, 포터블 exe 는 더블클릭으로 두 번 뜨기 쉬워 두 프로세스가 같은 설정 파일을 함께 쓰다 깨졌습니다.

선택 OS 로 갈리는 부분을 전부 한 파일에 모았습니다. 분기가 코드 곳곳에 흩어지면 「Windows 에서 왜 이러지」를 찾을 때 전체를 뒤져야 합니다. 자식 프로세스 환경변수가 가장 까다로웠습니다 — Windows 의 환경변수는 대소문자를 구분하지 않는데, 코드에서 객체로 복사하는 순간 그 성질이 사라져 Path 와 PATH 가 다른 항목으로 갈립니다. 하나로 합칩니다 GUI 로 띄운 앱은 로그인 시점의 PATH 를 물려받습니다 — CLI 를 방금 설치했으면 아직 PATH 에 없습니다. 그래서 흔한 설치 위치들을 앱이 직접 뒤지고, 그래도 없으면 설치 방법을 안내합니다 중복 실행을 막습니다 — 두 프로세스가 같은 설정 파일을 쓰면 깨집니다. 두 번째 프로세스는 즉시 죽고 폴더 경로만 원본 프로세스로 넘깁니다. 터미널에서 claude-talk 을 칠 때 이미 떠 있는 앱에서 방이 열리는 것도 같은 장치입니다

결과 macOS dmg 와 Windows 설치본·포터블 exe 를 같은 코드에서 냅니다. mac 에서 별도 도구 없이 Windows 산출물까지 빌드됩니다. OS 분기를 찾을 때 파일 한 개만 봅니다.

결과

- 제가 매일 쓰는 도구가 됐습니다. 여섯 방을 띄워둬도 방 이름으로 무슨 작업인지 알고, 끝나면 알림이 와서 잊고 있다가 뒤늦게 돌아가는 일이 없어졌습니다.

- 전제를 버려서 문제의 클래스를 없앴습니다 — TUI 스크래핑을 헤드리스 stream-json 으로 바꾸자 v1 의 방어 코드(재전송 안전망·준비 감지·신뢰 다이얼로그 회피)가 한 줄도 남지 않았습니다.
- 제 수정을 실측으로 뒤집었습니다 — 압축 임계 180k 는 추측에서 나온 값이었고, 압축 기록 37건을 재보니 소요시간이 크기와 무관해 판단이 반대로 뒤집혔습니다. 두 선택의 근거를 코드에 남겼습니다.
- 비용을 숫자로 봤습니다 — 요청당 평균 컨텍스트 89만, 캐시 읽기가 하루 비용의 78%. 「느낌상 비싸다」로는 임계를 정할 수 없었습니다.
- 내 앱 안에서만 맞는 상태를 만들지 않았습니다 — 기록 파일 하나 = 방 하나, 제목·삭제·이름 변경까지 터미널 claude --resume 목록과 1:1로 맞췄습니다.
- 발열은 프레임워크 탓이 아니었습니다 — content 를 애니메이션하던 점 세 개가 문서 전체를 초당 3~4회 재배치시키고 있었습니다. opacity 로 바꾸고 격리했습니다.
- macOS · Windows 를 같은 코드에서 냅니다. OS 분기는 파일 한 개에 모여 있습니다.
- 아직 v0.1.0 입니다. 배포보다 제 작업 흐름에서 실제로 쓰이는지를 먼저 봤고, 그 과정에서 나온 문제들을 위에 적었습니다.

일하는 방식

원인 레이어까지 내려간다

화면에서 시작해도 원인이 프로토콜·서버·빌드 단계에 있으면 그곳까지 내려가 고칩니다. 시세 위젯의 대역폭 문제는 프론트가 아니라 프록시가 압축 헤더를 걷어낸 탓이어서 WS 프로토콜을 다시 설계했고, 소스 탐색은 재파싱 대신 컴파일러가 이미 계산한 위치를 가로채는 쪽으로 내려가 해결했습니다.

재사용 가능한 구조로 남긴다

한 번 고친 문제는 일회성 패치가 아니라 모듈로 남깁니다. react-code-locator는 훅 하나를 Turbopack까지 6종 번들러의 모듈 해석 지점에 꽂는 구조로 확장했습니다.

배포·운영까지 직접 끌고 간다

프로토타입에서 멈추지 않습니다. 시세 위젯은 크롬 웹스토어에, 로케이터는 npm에, 터널은 vsix 패키지로 실제 배포했고, 시세 위젯은 실사용자와 평점까지 관리하며 운영 중입니다.

도구는 쓰는 사람 자리에 둔다

기능이 아무리 좋아도 쓰러 가야 하면 안 씁니다. 시세는 보던 페이지 위에, 소스 점프는 화면 우클릭에, 터널은 에디터 사이드바에 뒀습니다. 하던 일을 멈추지 않는 게 기준이었습니다.

스킬

FRONTEND	React · TypeScript · JavaScript · Webpack · Vite · Chrome Extension (MV3)
실시간 · 프로토콜	WebSocket · SSE · 리버스 프록시
BACKEND · INFRA	Node.js · RESTful API 설계 · Render
TOOLING	번들러 플러그인 (Vite·webpack·Turbopack) · Babel·AST · npm 패키징 · VSCode Extension API

스택을 가리지 않고
원인까지 내려가는 사람을 찾고 계신다면.

github.com/dbckdgjs369 dbckdgjs1130@naver.com 010-2809-6995